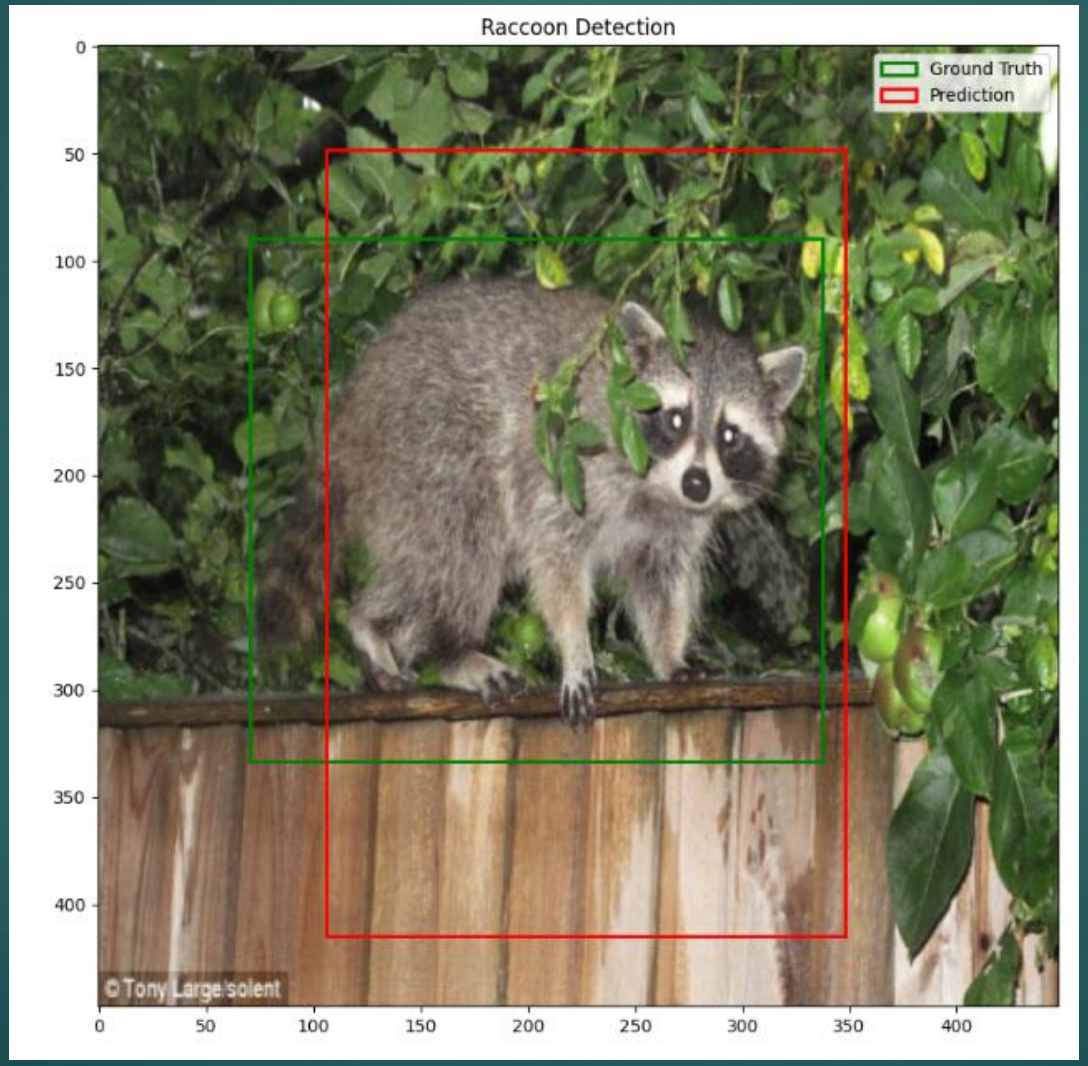
Intro to Object Detection

EDINBURGH AI

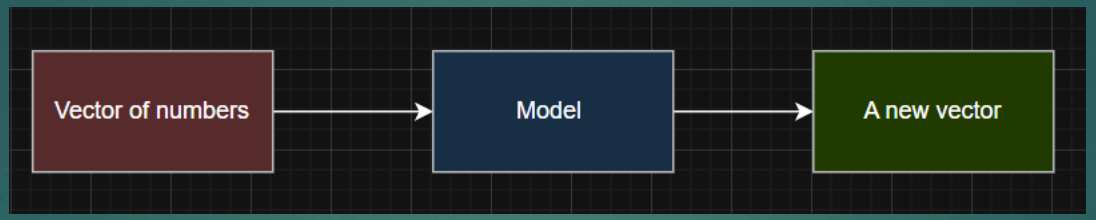


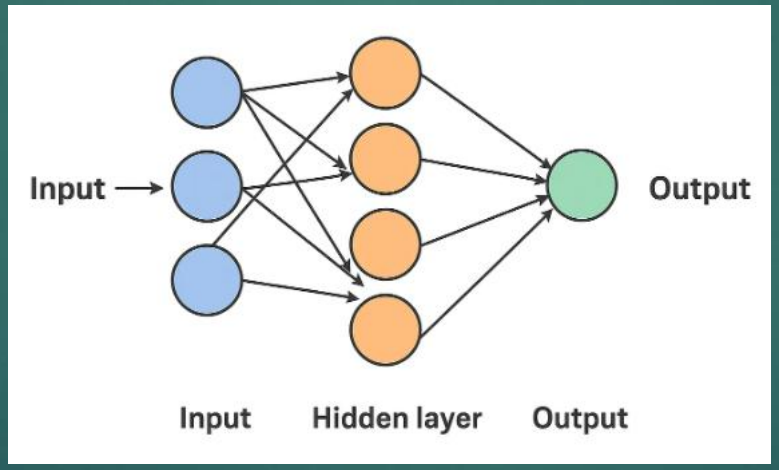
What we'll be building today



Recap:
What is a Neural Network?

Neural Network overview





If neural networks take in numbers, how do we input an image into one?

How can we apply a neural network to images?

At its core, an image on a computer is just a matrix of pixels

- Each pixel has a Red, a green and a blue value. Together they make a colour.

210, 99, 4	63, 22, 180	255, 12, 12
190, 45, 11	5, 150, 88	112, 44, 90
200, 91, 104	18, 18, 18	156, 32, 200

Convolutional Neural Network

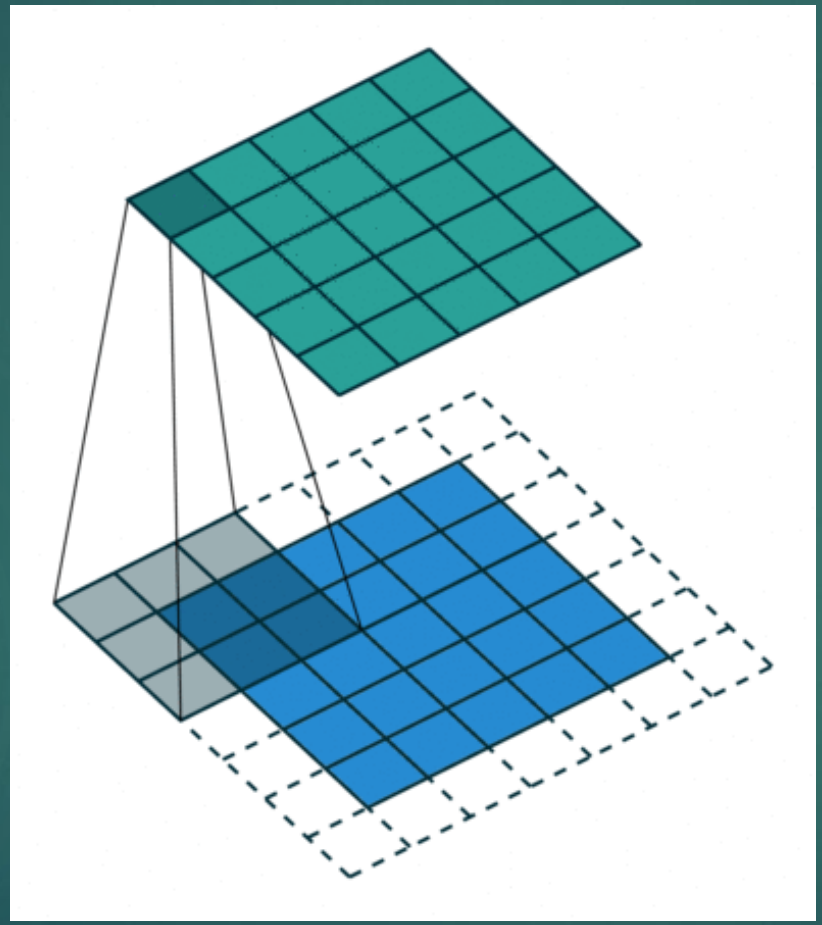
A convolutional is just a neural network that has layers that perform a convolution.

The equation we use for our convolution varies on the dimension of our input.

e.g.

- 1d convolution – used for audio waves, stock prices
- 2d convolution – used for 2d images

So, what is a convolution?

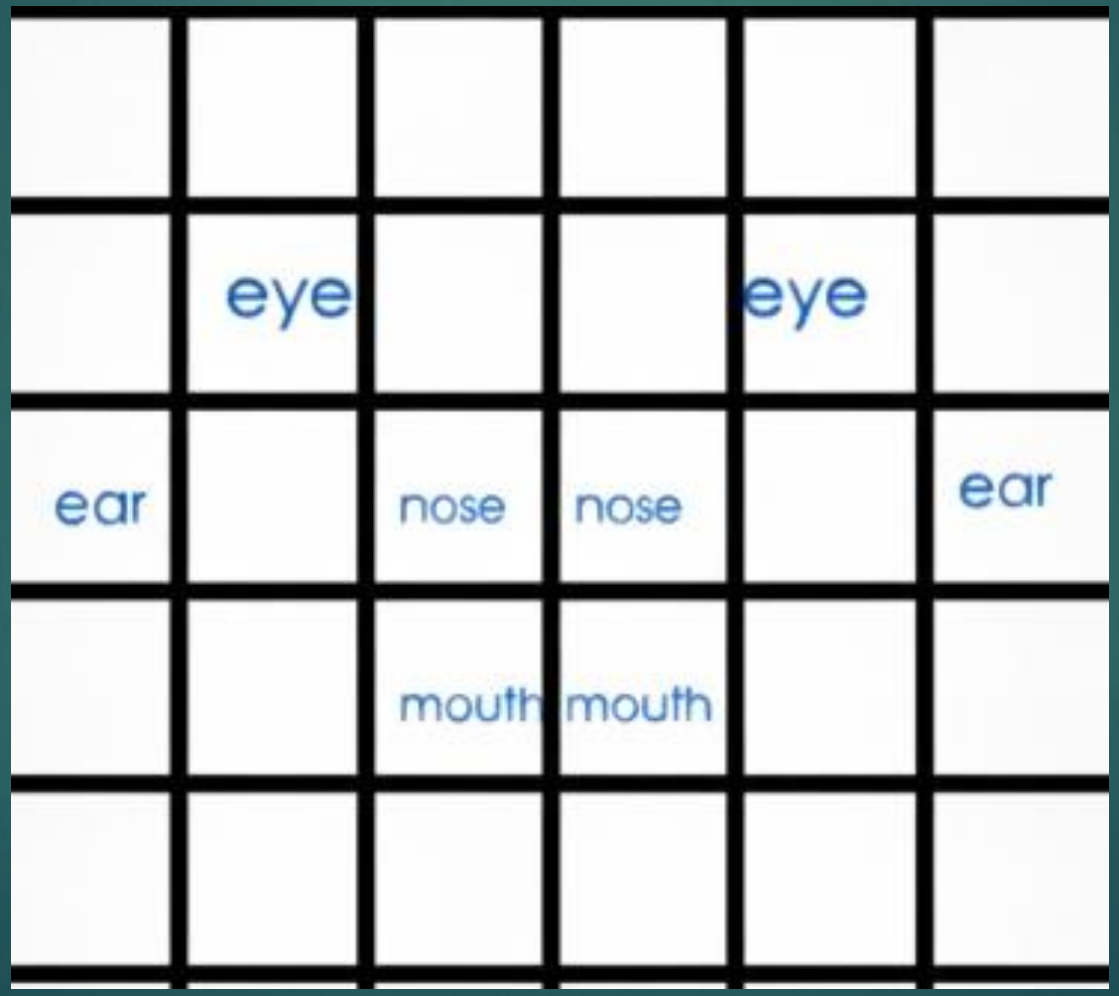


Why do we use convolutions for image processing

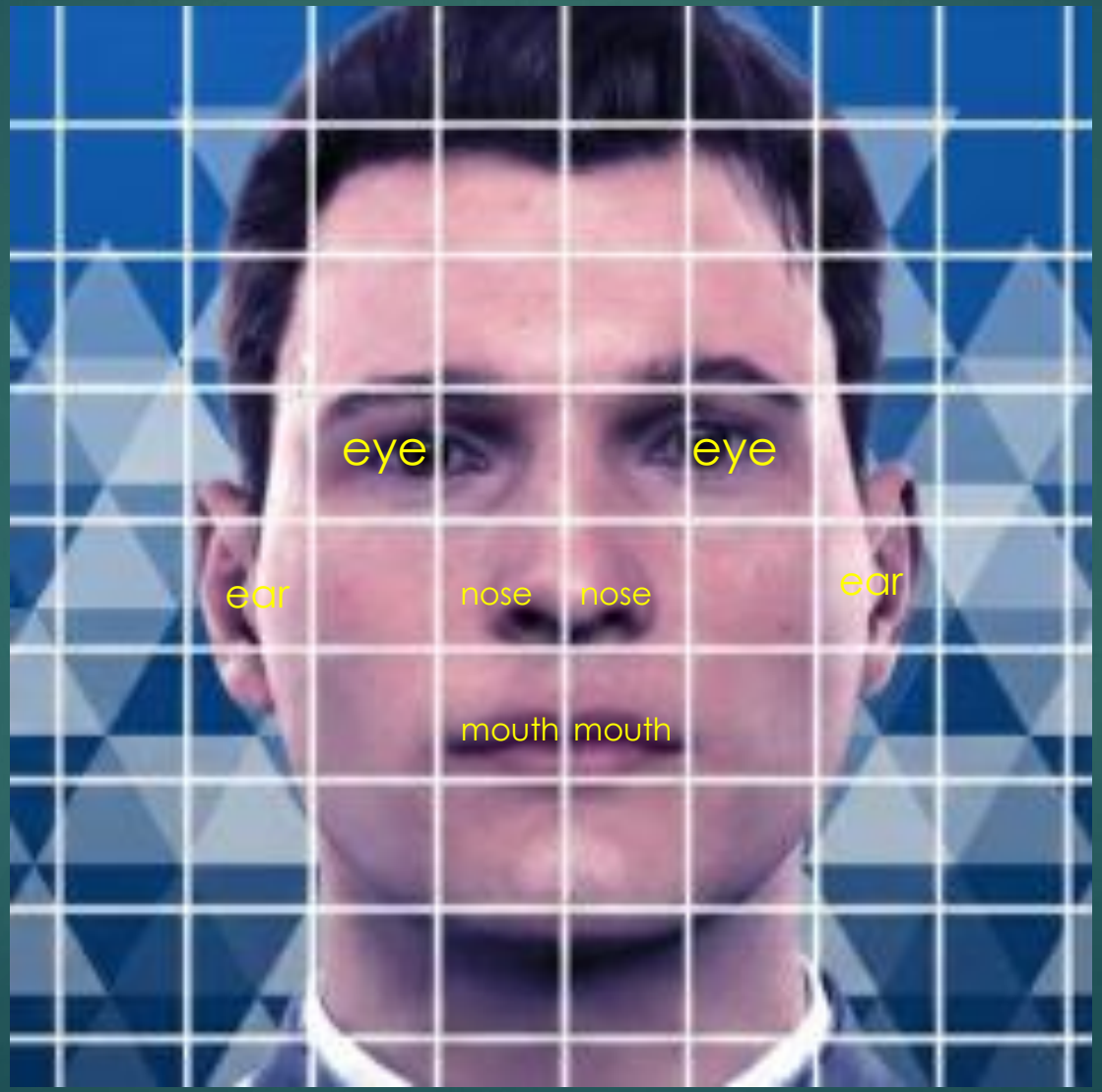
Convolutional neural networks are very good at being aware of not just what is in image, but also where.

If we know all the small features in an image and where they are, we can decipher what's in the image!

Example: What's in this image?



A person! (debatably)



How can a neural network classify what this is an image of

We make it so each element in our output vector corresponds to a prediction of what is in the image.

If the model believes the corresponding object is in the image it puts a 1 in its element, if it believes the image doesn't contain the corresponding object it instead puts a 0.

Example

If the elements of our output vector correspond to:

[dog, cat, horse, snake, rabbit]

If our model believes there is a snake in the image it should output

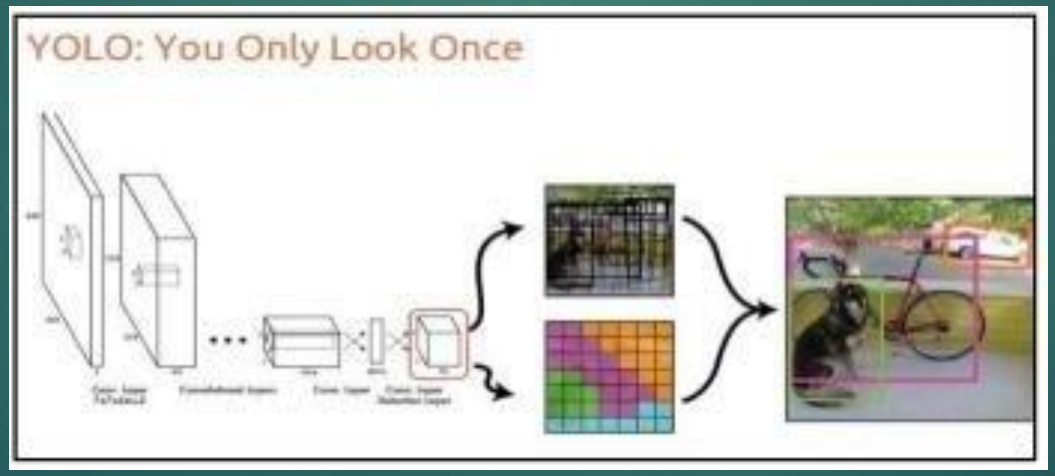
[0,0,0,1,0]

This is great for telling us what is in an image, but it doesn't tell us where

YOLO Architecture

What is YOLO?

In a deep learning context, YOLO (You Only Look Once) is a computer vision architecture used for object detection and location in images and videos.



YOLO makes one main simple change to our classification model

Instead of predicting a vector saying, "yes this is in the image", it instead predicts a vector containing:

[confidence, object_center_x, object_center_y, box_height, box_width]

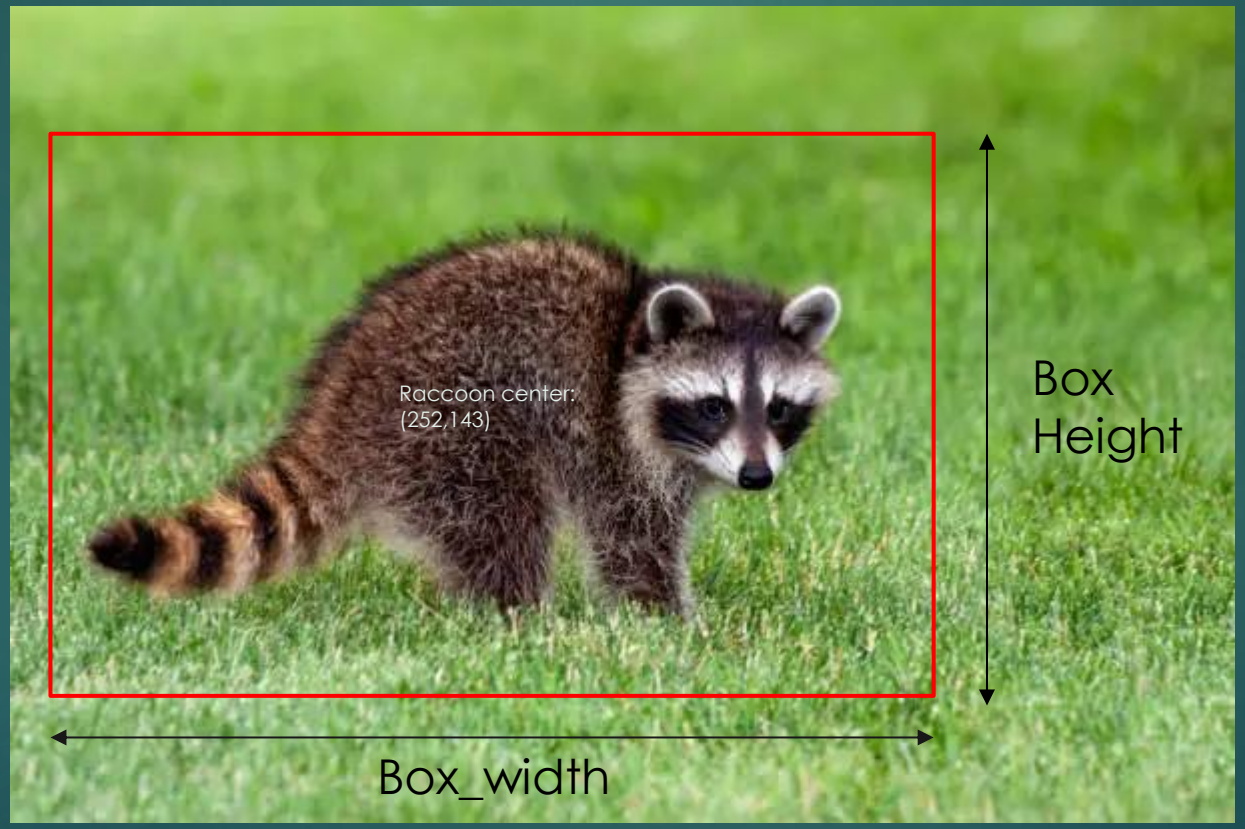
Would anyone like to take a guess at what these values mean?

What the elements mean

- **Confidence –** a value between 1 and 0 saying how confident the model is that the image contains the
- **object_center_x –** the x coordinate of the center of the object in the image
- **object_center_y –** the x coordinate of the center of the object in the image
- **box_width –** the width (in pixels) of the bounding box we need to draw around our object
- **box_height –** the height (in pixels) of the bounding box we need to draw around our object

Note: for now, we've moved away from being able to detect if multiple types of object are in the image and are now just predicting if 1 type of object is. Later versions of YOLO handle multiple objects but they're a bit more complex.

What this looks like on an image



Are there any implications of changing our output vector?

Classification CNN loss function

For the classification CNN, the only thing the model could get wrong is what the image was.

We therefore used a loss function such as Mean Squared Error that punished the model for putting a 1 in the wrong unit in the output vector.

(where n = batch size)

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

YOLO Loss function

Since for YOLO we're predicting both if the object is in the image and where it is, there's effectively now 2 things it can get wrong:

- It can give a wrong prediction on if the object is in the image or not
- It can give a wrong estimate for where the object is in the image

However, if the image doesn't actually have the object in it, it is unfair to punish the coordinate loss and instead we only punish the confidence loss

YOLO Loss function - The Maths

You can see a breakdown of it here if you like:

[insert link to obsidian notes here]

For a single data point:

$$\text{Total loss} = \text{coordinate loss} + \text{confidence loss}$$

$$= (\hat{c} - c)^2 + [(x - \hat{x})^2 + (y - \hat{y})^2] + [(\sqrt{w} - \sqrt{\hat{w}})^2 + (\sqrt{h} - \sqrt{\hat{h}})^2]$$

For a batch of size n:

$$\text{Total loss} = \frac{1}{n} \sum_{i=1}^{n} \left((\hat{c}_i - c_i)^2 + [(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2] + \left[(\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right] \right)$$

Evaluating our YOLO model

In practice, no matter how good our model is, it's never going to get the bounding boxes absolutely perfect every time.

We need a way of deciding if our model's bounding box is close enough to the target.

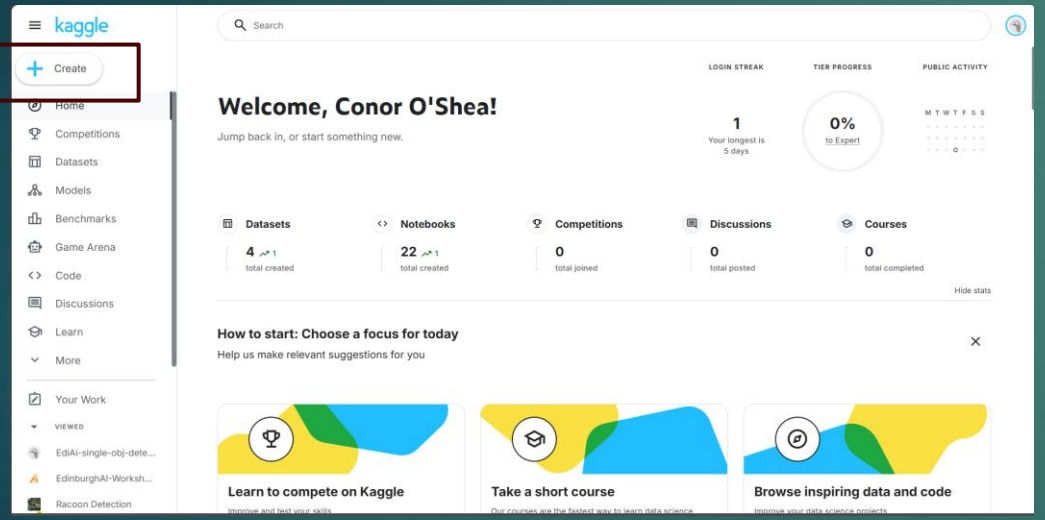
Any ideas?

Intersection over union

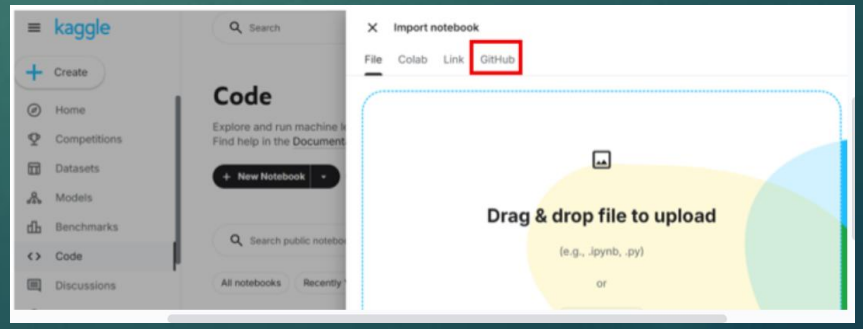
Now it’s time for you to try it yourselves!

Kaggle Setup

1. Go to: https://www.kaggle.com/
2. Create an account (if you don't have one already)
3. Click the create button in the top left and select "Import Notebook" from the dropdown menu.



5. Select "GitHub"
6. In the search prompt field enter: "EdinburghAI/workshops"



Kaggle set up - 2